파이썬 완전 백과사전 2권 · 딥러닝 기초 교안

4일차

CNN: 돋보기를 옮기며 무늬 찾기

16~20차시 · 책 9장 CNN과 이미지 분류

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/09-cnn (5일 과정의 확장 첫날)

오늘의 시간표

차시	주제	책 범위	오래 걸리는 예제
16	합성곱: 돋보기를 옮기며 무늬 찾기	9장 들어가며 ~ 9.2	없음 (모두 1초 안팎)
17	채널, 파라미터 공유, 풀링	9.3 ~ 9.4	없음
18	첫 CNN, 그림을 밀어도 알아볼까	9.5 ~ 9.6	b10 학습 113초
19	배치 정규화, 유명한 구조, CNN이 보는 것	9.7 ~ 9.9	b20 비교 8초
20	실습: 작은 ResNet으로 다시 만들기	9.10	b24 학습 66초 (GPU)

한 차시 50분: 도입 5분 · 설명과 시연 30분 · 직접 실행 10분 · 정리 5분 · 오래 걸리는 학습은 차시 첫머리에 실행을 걸어 둔다

준비: 09-cnn 폴더로 이동 · 18차시 전에 6장의 fashion_mlp.pt를 이 폴더로 복사 · FashionMNIST는 첫 실행 때 data/로 내려받는다

16차시

16

합성곱: 돋보기를 옮기며 무늬 찾기

책 9장 들어가며 ~ 9.2절

학습 목표

- 사진을 한 줄로 펴면 생기는 두 문제를 숫자로 말한다
- 합성곱을 손으로 계산하고 nn.Conv2d와 맞춰 본다
- 패딩과 보폭을 넣은 출력 크기를 공식으로 구한다

9장 사진을 한 줄로 펴면 무엇을 잃을까

연결이 너무 많다

입력 784칸 하나하나를 은닉층 모든 칸에 따로 연결한다. 6차시에서 센 것처럼 첫 층이 파라미터의 98% 넘게 차지했다

자리에 묶여 있다

한 줄로 편 입력은 "몇 번째 칸에 잉크가 있나"만 알지 "어떤 모양이 있나"는 모른다. 그림을 밀면 무너진다(18차시에서 잰다)

사진 (채널×세로×가로)	완전 연결층 (256칸)	합성곱층 (3×3 필터 256개)
옷 사진 1×28×28	200,960개 (0.77 MB)	2,560개 (0.01 MB)
작게 줄인 컬러 사진 3×224×224	**38,535,424개 (147.00 MB)**	**7,168개 (0.03 MB)**

16차시 · 9장 들어가며 · 예제 09-cnn/b01_fc_params.py (출력을 표로)

9.1 손으로 하는 합성곱

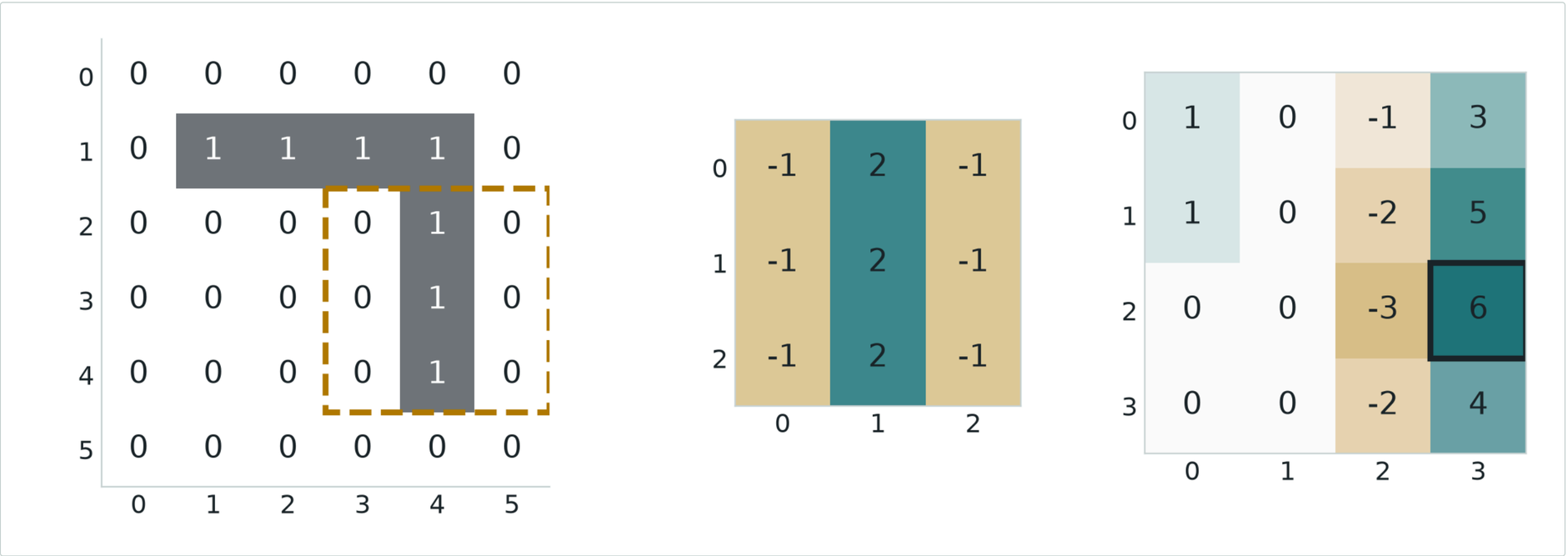
신문 위에서 돋보기를 한 칸씩 옮기며 찾는 글자가 있는지 본다. 돋보기 = **필터(커널)**, 댈 때마다 "같은 자리끼리 곱해서 모두 더한 값"을 적는다

```
# 3×3 돋보기(필터): 가운데 세로줄은 +2, 양옆은 -1 → 세로선이 있으면 크게 반응
필터 = torch.tensor([
    [-1.0, 2.0, -1.0],
    [-1.0, 2.0, -1.0],
    [-1.0, 2.0, -1.0],
])
k = 필터.shape[0]
결과 = torch.zeros(6 - k + 1, 6 - k + 1)          # 돋보기를 놓을 수 있는 자리: 4×4
for i in range(6 - k + 1):                          # 위에서 아래로
    for j in range(6 - k + 1):                      # 왼쪽에서 오른쪽으로
        조각 = 그림[i:i + k, j:j + k]               # 돋보기 아래의 3×3
        결과[i, j] = (조각 * 필터).sum()            # 같은 자리끼리 곱해서 모두 더하기
```

16차시 · 9.1 합성곱: 돋보기를 옮기며 무늬 찾기 · 예제 09-cnn/b02_hand_conv.py (핵심 부분)

9.1 결과 표 = 특징 맵

그림 'ㄱ' 6×6 | 필터 3×3 | 특징 맵 4×4



가장 크게 반응한 자리 (2, 3): 돋보기(점선) 한가운데가 세로획 아랫부분에 놓인다 · 청록 = 양수, 노랑 = 음수

9.1 F.conv2d와 nn.Conv2d

```
# 4차원으로: 그림 (장 수, 채널, 세로, 가로), 필터 (출력 채널, 입력 채널, 세로, 가로)
x = 그림.reshape(1, 1, 6, 6)
w = 필터.reshape(1, 1, 3, 3)
print(x.shape, "*", w.shape, "→", F.conv2d(x, w).shape)
층 = nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, bias=False)
print("층의 필터 모양:", 층.weight.shape)
with torch.no_grad():
    층.weight.copy_(w)                    # 무작위 필터를 우리 필터로 바꿔 끼우기
print("손 계산과 같은가?", torch.equal(층(x), F.conv2d(x, w)))
```

```
torch.Size([1, 1, 6, 6]) * torch.Size([1, 1, 3, 3]) → torch.Size([1, 1, 4, 4])
층의 필터 모양: torch.Size([1, 1, 3, 3])
손 계산과 같은가? True
```

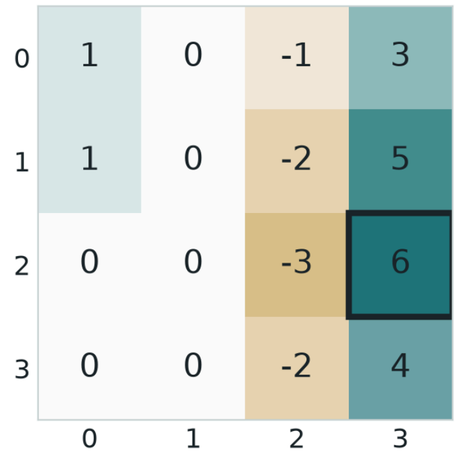
필터 값은 사람이 정하지 않는다: **어떤 무늬를 찾을지를 학습으로 정한다**

16차시 · 9.1 F.conv2d와 nn.Conv2d · 예제 09-cnn/b03_f_conv2d.py (핵심 부분)

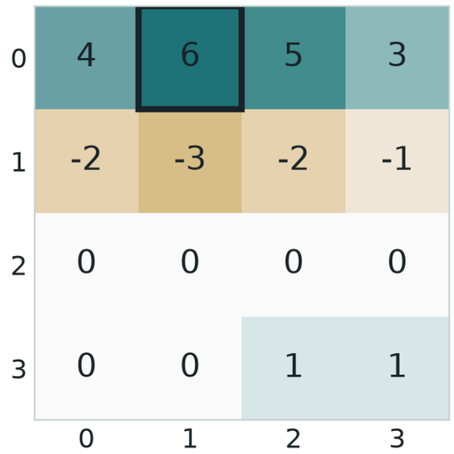
9.1 필터가 여러 개면 특징 맵도 여러 장

```
필터들 = torch.stack([세로선, 가로선, 모서리]).unsqueeze(1)   # (3, 1, 3, 3): 필터 3개
특징맵 = F.conv2d(그림, 필터들)                                # (1, 3, 4, 4): 필터마다 한 장
```

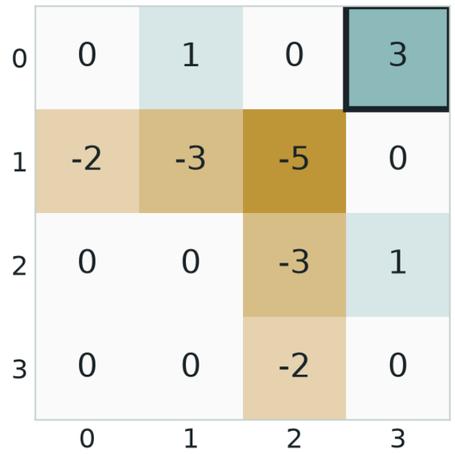
세로선 · 최대 6, (2, 3)



가로선 · 최대 6, (0, 1)



모서리 · 최대 3, (0, 3)



16차시 · 9.1 필터가 여러 개면 특징 맵도 여러 장 · 예제 09-cnn/b04_filters.py (핵심 부분과, 같은 계산을 칠한 그림)

9.2 패딩과 보폭: 출력 크기 정하기

```
def 출력_크기(n: int, k: int, p: int = 0, s: int = 1) -> int:
    """입력 n, 필터 k, 패딩 p, 보폭 s일 때 출력의 한 변 길이."""
    return (n + 2 * p - k) // s + 1
```

```
{'kernel_size': 3}                              → 26×26  (공식: 26)
{'kernel_size': 3, 'padding': 1}                → 28×28  (공식: 28)
{'kernel_size': 5}                              → 24×24  (공식: 24)
{'kernel_size': 5, 'padding': 2}                → 28×28  (공식: 28)
{'kernel_size': 3, 'stride': 2, 'padding': 1} → 14×14  (공식: 14)
{'kernel_size': 2, 'stride': 2}                 → 14×14  (공식: 14)
{'kernel_size': 3, 'padding': 'same'}           → 28×28  (공식: 28)
```

패딩: 액자 테두리

둘레에 0을 둘러 돋보기 가운데가 가장자리까지 닿게 한다

보폭: 돋보기의 걸음 폭

두 칸씩 건너뛰며 옮기면 출력이 절반으로 준다

16차시 · 9.2 패딩과 보폭 · 예제 09-cnn/b05_padding.py (핵심 부분과 출력, 입력은 28×28 옷 사진 크기)

9.2 출력 크기 공식과 자주 쓰는 세 조합

출력 = (입력 + 2 × 패딩 − 필터) ÷ 보폭 (몫만) + 1

설정	28×28 입력	언제 쓰나
kernel_size=3, padding=1	28×28 그대로	크기를 유지하며 무늬 찾기. 가장 흔한 조합
kernel_size=3, stride=2, padding=1	14×14 (절반)	무늬를 찾으면서 크기를 절반으로
kernel_size=5, padding=2	28×28 그대로	넓은 돋보기. 요즘은 3×3 두 번으로 대신 (19차시)

크기 유지: 홀수 필터 k에 패딩 (k − 1) ÷ 2, 또는 **padding="same"** (보폭 1일 때만)

직접 실행 10분

1. b02_hand_conv.py와 b03_f_conv2d.py를 실행해 반복문 결과와 F.conv2d 결과가 같은지 본다
2. b02의 그림을 'ㄴ'으로 바꾼다 (1행~3행은 1열만 1, 4행은 1~4열이 1). 세로선 필터는 어디서 가장 크게 반응할까? 먼저 예상하고 실행한다
3. b05_padding.py의 설정들에 dict(kernel_size=7, stride=2, padding=3)을 더한다. 공식으로 먼저 계산하고 실행해 맞춰 본다

```
cd examples/vol2/09-cnn && uv run python b02_hand_conv.py
```

32×32 사진에 nn.Conv2d(1, 1, kernel_size=5)를 대면 출력은?

Q

가 32×32

나 30×30

다 28×28

라 16×16

16차시 정리

합성곱은 작은 필터를 옮겨 가며 같은 자리끼리 곱하고 모두 더하는 계산, 결과는 무늬의 위치 지도(**특징 맵**)

nn.Conv2d는 필터를 파라미터로 품는다. **어떤 무늬를 찾을지는 학습으로 정한다**

출력 = (입력 + 2 × 패딩 − 필터) ÷ 보폭 + 1. 유지는 3×3에 **padding=1**, 절반은 **stride=2**

다음 차시: 컬러 사진처럼 겹친 그림(채널)과, 지도를 줄여 요약하는 풀링

16차시 · 정리 · 복습: 책 9장 들어가며 ~ 9.2절

17차시

17

채널, 파라미터 공유, 풀링

책 9.3 ~ 9.4절

학습 목표

- 필터 묶음 (출력 채널, 입력 채널, 세로, 가로)을 읽고 파라미터를 센다
- 파라미터 공유와 1×1 합성곱이 하는 일을 말한다
- 최대·평균 풀링과 전역 평균 풀링의 결과를 예측한다

9.3 필터도 채널만큼 두껍다

```
컬러 = torch.rand(1, 3, 5, 5)                    # 컬러 사진 1장: 빨강·초록·파랑 3채널, 5×5
층 = nn.Conv2d(in_channels=3, out_channels=8, kernel_size=3)
print("필터 묶음:", 층.weight.shape, " 편향:", 층.bias.shape)
print("파라미터:", sum(p.numel() for p in 층.parameters()), "=", 8 * 3 * 3 * 3 + 8)
출력 = 층(컬러)
# 출력 0번 채널의 왼쪽 위 칸을 손으로: 세 채널의 3×3을 각자 곱해 모두 더하고 편향
조각 = 컬러[0, :, 0:3, 0:3]                     # (3, 3, 3): 세 채널의 왼쪽 위 3×3
손으로 = (조각 * 층.weight[0]).sum() + 층.bias[0]
```

```
필터 묶음: torch.Size([8, 3, 3, 3])  편향: torch.Size([8])
파라미터: 224 = 224
출력: torch.Size([1, 8, 3, 3])
손으로 0.111630, 층이 낸 값 0.111630
```

- (8, 3, 3, 3) = 필터 8개, 하나는 **3채널 × 3 × 3**
- 한 자리에서 **27개**를 곱해 모두 더하고 편향

17차시 · 9.3 채널: 겹겹이 쌓인 그림 · 예제 09-cnn/b06_channels.py (핵심 부분)

9.3 합성곱층의 파라미터 세기

출력 채널 × 입력 채널 × 필터 세로 × 필터 가로 + 출력 채널

nn.Conv2d(3, 8, kernel_size=3): 8 × 3 × 3 × 3 + 8 = **224**

사진 크기는 들어가지 않는다

입력이 5×5든 1000×1000이든 224개 그대로

16차시의 표가 이 공식

흑백 256 × 1 × 9 + 256 = 2,560
컬러 256 × 3 × 9 + 256 = 7,168

9.3 같은 돋보기를 어디서나: 파라미터 공유

```
# 28×28 사진 → 28×28 지도 32장을 만드는 두 방법
합성곱 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
완전연결 = nn.Linear(28 * 28, 32 * 28 * 28)
```

합성곱

320개

완전 연결

19,694,080개

출력 모양은 같은데 **6만 배** 넘게 차이. "세로선은 사진 왼쪽에 있든 오른쪽에 있든 세로선"이라는 성질을 구조에 넣어 두었다

9.3 1×1 합성곱: 자리마다 채널만 섞기

```
x = torch.randn(2, 64, 7, 7)                        # 2장, 채널 64, 7×7
일대일 = nn.Conv2d(64, 16, kernel_size=1)
선형 = nn.Linear(64, 16)                            # 같은 가중치를 복사해 넣었다
자리마다 = 선형(x.permute(0, 2, 3, 1)).permute(0, 3, 1, 2)   # 채널을 맨 뒤로 보내 Linear, 다시 제자리
print("자리마다 Linear와 같은가?", torch.allclose(일대일(x), 자리마다, atol=1e-6))
```

```
1×1 합성곱: torch.Size([2, 64, 7, 7]) → torch.Size([2, 16, 7, 7]) 파라미터 1040
자리마다 Linear와 같은가? True
```

주변은 보지 않고 **한 자리에서 채널끼리만** 섞는다

채널 수를 싸게 늘리고 줄이는 도구: 64 × 16 + 16 = **1,040**

9.4 풀링: 줄여서 요약하기

골목 하나하나보다 "이 구역에 편의점이 있다"가 쓸모 있을 때가 많다. 지도를 2×2 구역으로 나눠 숫자 하나만 남긴다

특징 맵 4×4

```
1  3  0  2
4  2  1  0

0  1  5  6
2  0  7  1
```

```
최대 풀링: [[4.0, 2.0], [2.0, 7.0]]
평균 풀링: [[2.5, 0.75], [0.75, 4.75]]
한 칸 움직여도 같은가? True
```

- MaxPool2d(2): 구역에서 가장 강한 반응만. 왼쪽 위 [1, 3, 4, 2] → 4
- AvgPool2d(2): 구역의 평균. 같은 구역 → 2.5
- 9가 (0, 0)에서 (1, 1)로 옮겨도 같은 구역이면 결과가 같다

9.4 전역 평균 풀링: 채널마다 평균 하나

```
특징맵 = torch.randn(8, 128, 7, 7)                # 8장, 채널 128, 7×7
전역 = nn.AdaptiveAvgPool2d(1)(특징맵)
print(특징맵.shape, "→", 전역.shape)
print("채널 평균과 같은가?", torch.allclose(전역.flatten(1), 특징맵.mean(dim=(2, 3))))
print("14×14를 넣어도:", nn.AdaptiveAvgPool2d(1)(torch.randn(8, 128, 14, 14)).shape)
```

```
torch.Size([8, 128, 7, 7]) → torch.Size([8, 128, 1, 1])
채널 평균과 같은가? True
14×14를 넣어도: torch.Size([8, 128, 1, 1])
```

지도 128장을 숫자 128개로: **"어디에"는 잊고 "얼마나"만** 남긴다. 지도 크기와 상관없다

직접 실행 10분

1. b06_channels.py의 층을 nn.Conv2d(3, 16, kernel_size=5)로, 조각을 컬러[0, :, 0:5, 0:5]로 바꾼다. 파라미터 수와 출력 모양을 먼저 예상하고 실행한다
2. b08_pool.py에서 나의 9를 (1, 1) 대신 (0, 2)에 둔다. "한 칸 움직여도 같은가?"는 어떻게 바뀔까?
3. b07_param_share.py의 1×1 합성곱을 16채널 대신 32채널로 바꾼다 (Linear와 reshape도 함께). 파라미터는?

```
uv run python b06_channels.py
```

nn.Conv2d(16, 32, kernel_size=3)의 파라미터는 몇 개일까요?

Q

가 288

나 4,608

다 4,640

라 입력 사진 크기를 알아야 셀 수 있다

17차시 정리

필터 묶음은 (**출력 채널, 입력 채널, 세로, 가로**), 필터 하나는 입력 채널만큼 두껍다

파라미터 공유: 같은 필터를 모든 자리에서 돌려 써서 사진 크기와 상관없다. 1×1 합성곱은 자리마다 채널만 섞는다

풀링은 줄여서 요약, **전역 평균 풀링**은 채널마다 평균 하나: "어디에"는 잊고 "얼마나"만

다음 차시: 부품을 모아 첫 CNN으로 옷 사진을 분류하고, 그림을 밀어 본다

18차시

18

첫 CNN, 그림을 밀어도 알아볼까

책 9.5 ~ 9.6절

학습 목표

- CNN을 지나는 텐서의 모양을 층마다 따라간다
- 작은 CNN을 학습해 8차시의 MLP와 클래스별로 비교한다
- 등변성과 불변성으로 밀기 결과를 설명한다

9.5 먼저 학습부터 걸어 두기

```
cd examples/vol2/09-cnn
cp ../06-nn-module/fashion_mlp.pt .            # 6장 MLP: 비교와 밀기 시험에 쓴다
uv run python b10_fashion_cnn.py             # 5에폭 학습, fashion_cnn.pt 저장
```

강의용 컴퓨터, CPU

113초

재현을 위해 씨앗을 고정하고 CPU로 돈다. 도는 동안 모양 따라가기와 옷도구.py를 본다

느리면 장치를 장치_고르기()로 바꿔 GPU로 (소수점 아래는 조금 달라진다)

9.5 모양을 따라가 보기

층	출력 모양	파라미터
입력	(64, 1, 28, 28)	
Conv2d	(64, 32, 28, 28)	320
ReLU	(64, 32, 28, 28)	0
MaxPool2d	(64, 32, 14, 14)	0
Conv2d	(64, 64, 14, 14)	18,496
ReLU	(64, 64, 14, 14)	0
MaxPool2d	(64, 64, 7, 7)	0
Flatten	(64, 3136)	0
Linear	(64, 128)	401,536
ReLU	(64, 128)	0
Linear	(64, 10)	1,290

크기는 줄이고 채널은 늘린다: 28 → 14 → 7, 채널 1 → 32 → 64

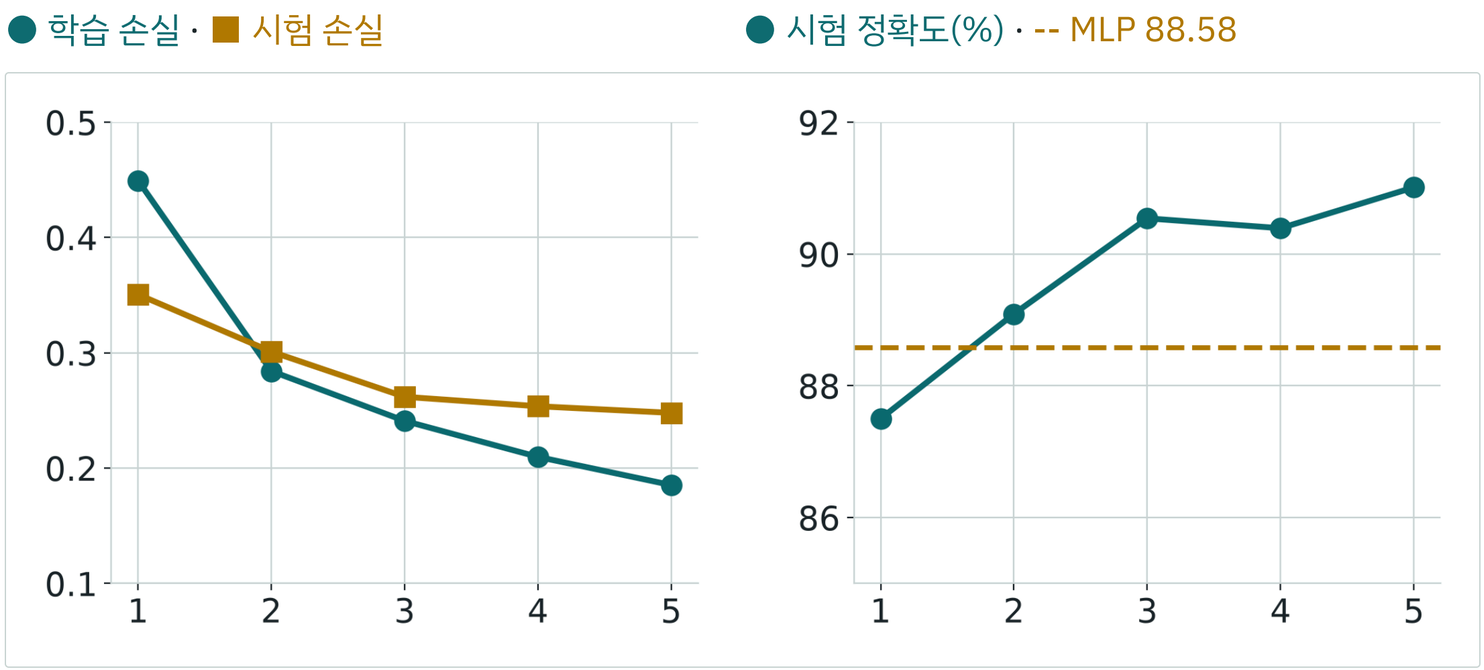
전체 **421,642개** 중 Linear(3136, 128) 하나가 **95%**. 무늬 찾기는 싸고, 펴서 연결하기가 비싸다

9.5 옷도구.py: 학습 코드는 모델만 다르다

```
class 옷CNN(nn.Module):
    """합성곱 두 단으로 무늬를 찾고, 완전 연결층 두 개로 옷 종류를 고르는 작은 CNN."""
    def __init__(self) -> None:
        super().__init__()
        self.특징 = nn.Sequential(                          # 그림 모양 그대로 무늬 찾기
            nn.Conv2d(1, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        )
        self.분류 = nn.Sequential(                          # 찾은 무늬로 종류 고르기
            nn.Flatten(), nn.Linear(64 * 7 * 7, 128), nn.ReLU(), nn.Linear(128, 10),
        )
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.분류(self.특징(x))
```

같은 파일에 8차시의 **한_에폭_학습 · 평가하기**, 6장 설계도 **의류분류기**, **시험_데이터 · 오른쪽으로_밀기**

9.5 5에폭 만에 91.01%



가로는 에폭 1~5

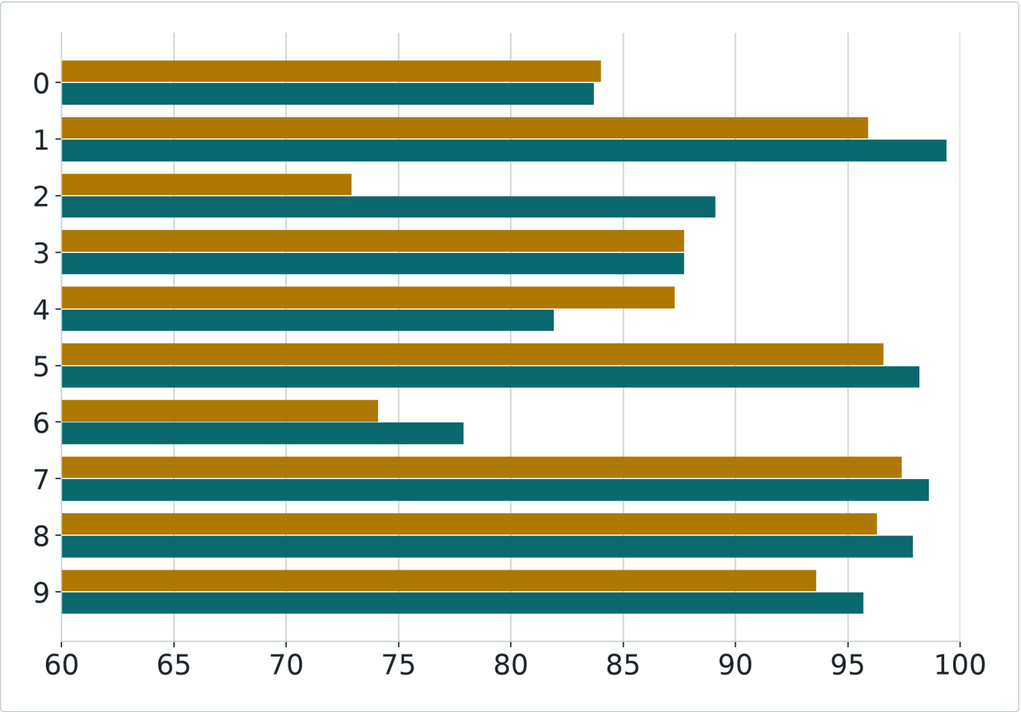
MLP는 10에폭에 88.58%, CNN은 **절반의 에폭으로 +2.4%p**

첫 에폭부터 **87.50%**

5에폭: 학습 0.1853, 시험 0.2479. 간격이 벌어진다: 과적합 대책은 아직 없다

18차시 · 9.5 학습시키기 · 책의 b10_fashion_cnn.py를 같은 설정(CPU, 씨앗 0)으로 다시 돌려 그린 그림

9.5 MLP와 무엇이 달라졌나



■ MLP 88.58% · ■ CNN 91.01% · 가로는 정확도(%)

0 T셔츠 1 바지 2 풀오버 3 드레스 4 코트 5 샌들 6 셔츠 7 운동화 8 가방 9 앵클부츠

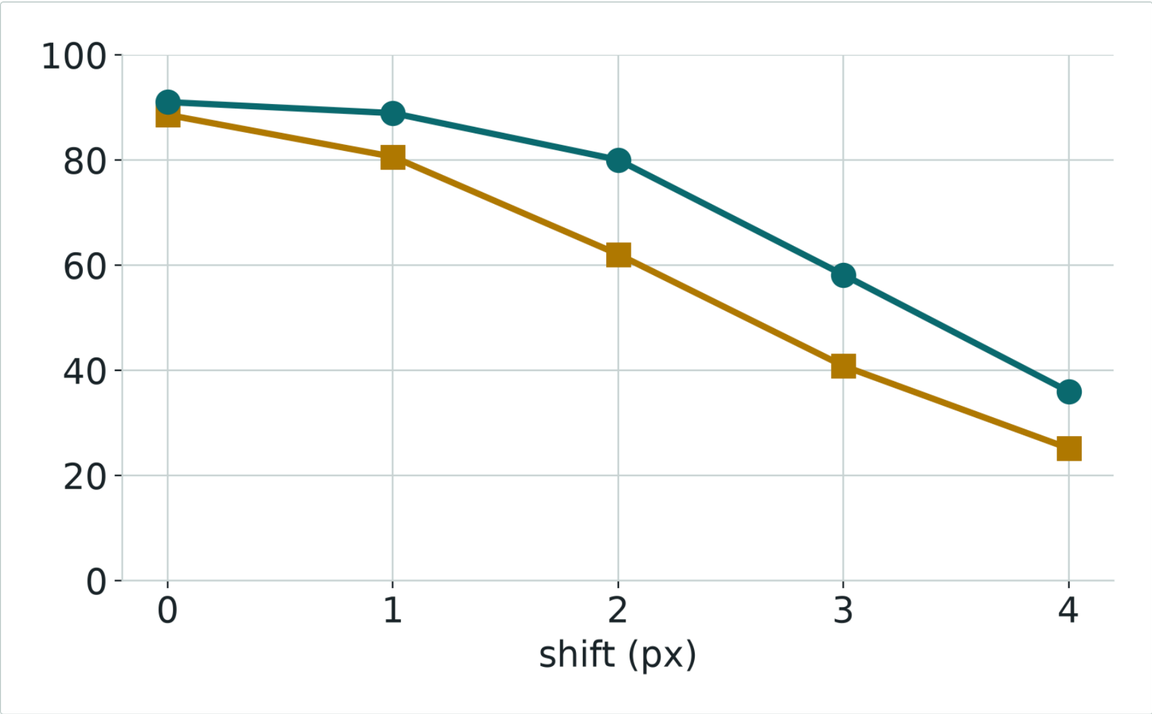
풀오버 72.9% → 89.1% (+16.2%p). "풀오버→코트 172번"이 헷갈림 순위에서 사라졌다

코트 87.3% → 81.9% (−5.4%p). "코트→풀오버 80번"이 새로 올라왔다

T셔츠↔셔츠는 둘 다 많이 헷갈린다: CNN도 T셔츠→셔츠 129번

18차시 · 9.5 MLP와 무엇이 달라졌나 · 책의 b11_compare.py와 같은 두 모델 파일로 계산한 그림

9.6 그림을 밀어도 알아볼까



■ MLP · ● CNN · 세로는 시험 정확도(%), 가로는 오른쪽으로 민 칸 수

칸	MLP	CNN
0	88.6%	91.0%
1	80.6%	88.9%
2	62.1%	80.0%
3	40.8%	58.2%
4	25.2%	36.0%

CNN은 훨씬 잘 버티지만 **세 칸부터 크게 무너진다**. "위치가 달라도 알아본다"는 반만 맞다

9.6 밀린 만큼 밀리는 합성곱, 자리에 묶인 Linear

1 합성곱: 등변

```
밀고 나서 합성곱
= 합성곱하고 밀기
True
```

무늬를 **찾는** 단계는 위치와 상관없이 공평하다

2 Flatten: 자리에 묶임

```
같은 칸끼리의 차이 0.0649
값 자체의 크기     0.2040
```

두 칸 밀리면 다음 Linear가 받는 입력이 크게 달라진다

3 전역 평균 풀링: 불변

```
밀기 전 [0.0826, -0.3031, ...]
밀고 나서 [0.0826, -0.3031, ...]
```

"어디에"를 버리니 밀어도 값이 같다

그래서 요즘 CNN은 Flatten + 큰 Linear 대신 **전역 평균 풀링 + 작은 Linear** (20차시)

직접 실행 10분

1. b10 학습이 끝났는지 보고 fashion_cnn.pt가 생겼는지 확인한 뒤, b11_compare.py와 b13_shift_test.py를 실행한다
2. b13을 복사해 오른쪽 대신 **아래로** 밀어 본다. 오른쪽으로 밀 때보다 더 무너질까, 덜 무너질까? 예상하고 실행한다

```
def 아래로_밀기(x: torch.Tensor, 칸: int) -> torch.Tensor:
    """사진 묶음을 아래로 칸만큼 옮기고 빈자리는 0으로 채운다."""
    밀린 = torch.zeros_like(x)
    밀린[..., 칸:, :] = x[..., :x.shape[-2] - 칸, :]
    return 밀린
```

b13의 오른쪽으로_밀기(X, 칸)을 아래로_밀기(X, 칸)으로 바꾸고, import에서 오른쪽으로_밀기를 뺀다

18차시 · 직접 실행 · b11, b13은 1초 안팎 · 09-cnn 폴더에 fashion_mlp.pt와 fashion_cnn.pt가 있어야 돈다

입력을 밀어도 출력이 바뀌지 않는(불변) 것은?

Q

가 nn.Conv2d(1, 4, 3, padding=1)의 출력

나 합성곱 뒤 Flatten한 결과

다 합성곱 뒤 AdaptiveAvgPool2d(1)한 결과

라 9.5절 CNN의 최종 점수

18차시 정리

CNN의 모양: **크기는 줄이고 채널은 늘린다**. 특징 부분과 분류 부분으로 나눠 담는다

학습 코드는 그대로, 모델만 바꿔 88.58% → 91.01%. 풀오버는 크게 좋아지고 코트는 나빠졌다

합성곱은 **등변**, Flatten + Linear는 **자리에 묶임**, 전역 평균 풀링은 **불변**

다음 차시: CNN 속 배치 정규화, 유명한 구조와 ResNet, CNN이 보는 것

19차시

19

배치 정규화, 유명한 구조, CNN이 보는 것

책 9.7 ~ 9.9절

학습 목표

- CNN 속 배치 정규화와 드롭아웃의 자리를 말한다
- VGG의 3×3 쌓기와 ResNet의 잔차 연결이 푼 문제를 설명한다
- 필터, 특징 맵, 입력 기울기로 CNN 안을 들여다본다

9.7 CNN 속의 배치 정규화와 드롭아웃

```
x = torch.randn(16, 3, 8, 8) * 5 + 2  # 16장, 채널 3, 8×8
bn = nn.BatchNorm2d(3)
# 채널마다 (장 수, 세로, 가로) 전체에서 평균·분산
평균 = x.mean(dim=(0, 2, 3), keepdim=True)
```

```
학습할 숫자: [(3,), (3,)]
손으로 계산과 같은가? True
채널별 평균: [0.0, 0.0, 0.0]
편향이 있으나 없으나 같은가? True
```

도구	놓이는 자리	비고
BatchNorm2d(채널)	합성곱 → 배치 정규화 → ReLU	합성곱은 bias=False
Dropout(p)	전역 평균 풀링(또는 Flatten) 뒤, 마지막 Linear 앞	합성곱 사이엔 잘 안 쓴다
데이터 증강	모델 밖, 학습 데이터의 transform	검증·시험에는 안 쓴다
weight decay 등	옵티마이저와 학습 루프	MLP와 똑같이

9.8 유명한 구조의 아이디어

모델	해	파라미터	합성곱층	float32	새로 가져온 아이디어
LeNet-5	1998	61,706	2		합성곱 → 풀링 → 완전 연결의 기본 틀
AlexNet	2012	61.1백만	5	233 MB	ReLU, 드롭아웃, 데이터 증강, GPU
VGG-16	2014	138.4백만	13	528 MB	3×3 필터만 깊게 쌓기
ResNet-18	2015	11.7백만	20	45 MB	잔차 연결로 수십~수백 층
ResNet-50	2015	25.6백만	53	97 MB	전역 평균 풀링으로 큰 Linear를 없앰

합성곱층이 VGG-16의 네 배인 ResNet-50이 오히려 **5분의 1 크기**. 깊이는 깊게, 크기는 작게

9.8 VGG: 3×3을 쌓는 이유

3×3을 2번 쌓았을 때, 출력 가운데 칸이 보는 입력

```
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 1 1 1 1 0 0 0 0 0
0 0 0 1 1 1 1 1 0 0 0 0 0
0 0 0 1 1 1 1 1 0 0 0 0 0
0 0 0 1 1 1 1 1 0 0 0 0 0
0 0 0 1 1 1 1 1 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
```

채널 64	파라미터	보는 범위
5×5 한 번	102,400	5×5
3×3 두 번	**73,728**	5×5
3×3 세 번		7×7

같은 범위를 **28% 적은** 파라미터로 보고, 사이에 ReLU가 한 번 더 들어간다. 출력 한 칸이 보는 범위 = **수용 영역**

9.8 ResNet: 고칠 양만 배우기

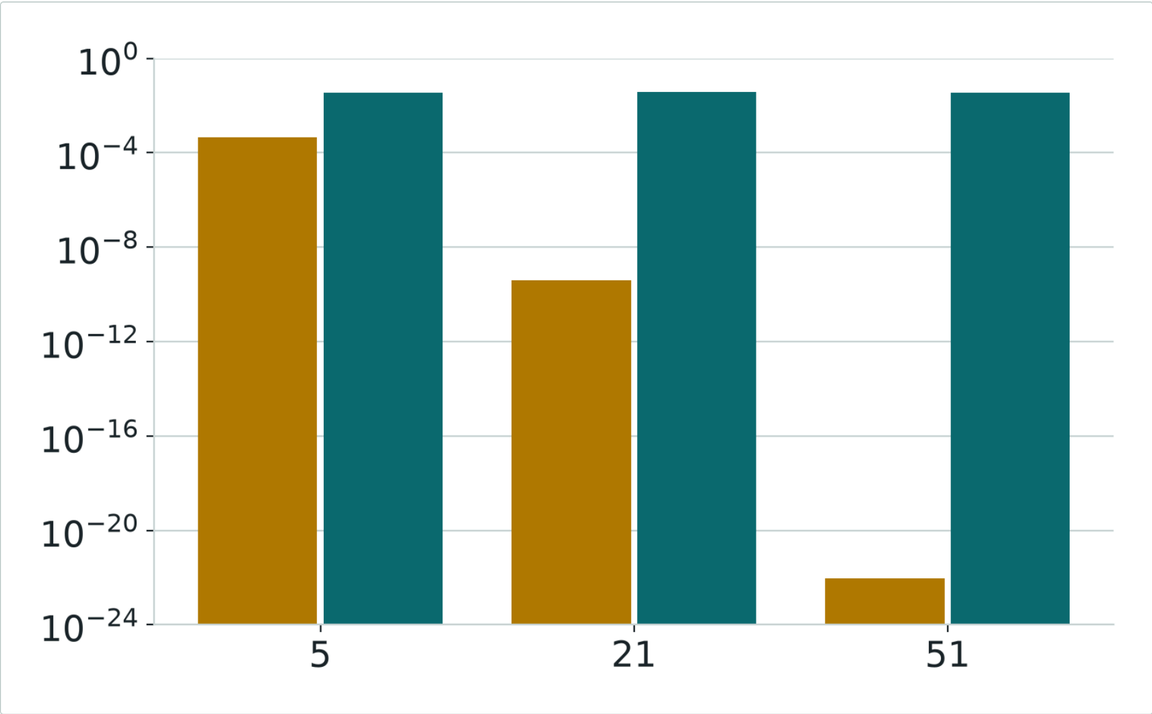
빈 종이에 다시 그리기보다 원래 그림에 **고칠 부분만** 덧칠하기. 블록은 $\mathbf{x} + \mathbf{F}(\mathbf{x})$를 내보낸다

```
class 잔차블록(nn.Module):
    # __init__: 합성곱1·정규화1·합성곱2·정규화2 (합성곱 → BN → ReLU 두 번)
    # 모양이 달라지면 지름길도 1×1 합성곱으로 모양을 맞춘다
    #   self.지름길 = nn.Sequential(nn.Conv2d(입력, 출력, 1, stride=보폭, bias=False), ...)
    # 아니면 self.지름길 = nn.Identity()
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        고칠양 = self.정규화2(self.합성곱2(F.relu(self.정규화1(self.합성곱1(x)))))
        return F.relu(고칠양 + self.지름길(x))          # 핵심은 이 덧셈 한 번
```

```
같은 크기: (8, 32, 14, 14) → (8, 32, 14, 14) Identity
줄이기   : (8, 32, 14, 14) → (8, 64, 7, 7) Sequential
고칠 양이 0이면 그대로 통과: True
```

19차시 · 9.8 ResNet: 고칠 양만 배우기 · 예제 09-cnn/b18_resblock.py (핵심 부분, __init__은 주석으로 줄임)

9.8 깊어지면 기울기가 사라진다

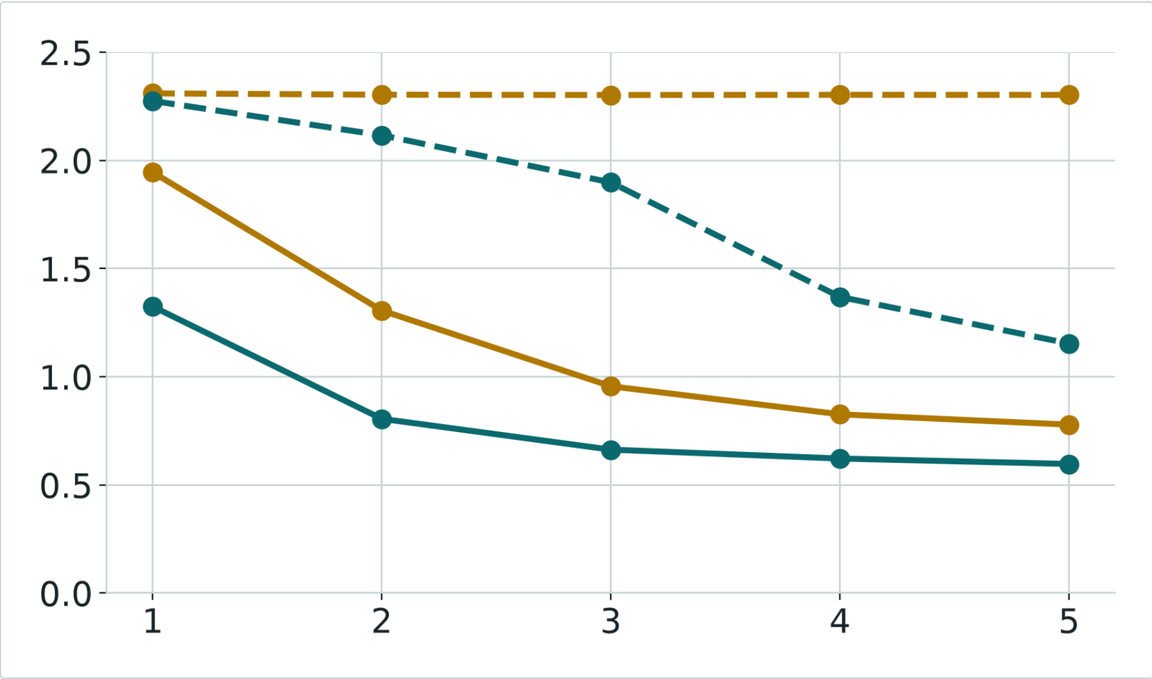


■ 평범한 CNN · ■ 잔차 CNN · 가로는 합성곱층 수, 세로는 첫 층 기울기 크기(로그)

층	평범한	잔차
5	4.48e-04	3.54e-02
21	3.94e-10	3.62e-02
51	9.17e-23	3.58e-02

x + F(x)를 x로 미분하면 **1 + (F의 기울기)**. 지름길로 기울기가 그대로 흐르는 고속도로

9.8 21층을 실제로 학습해 보면

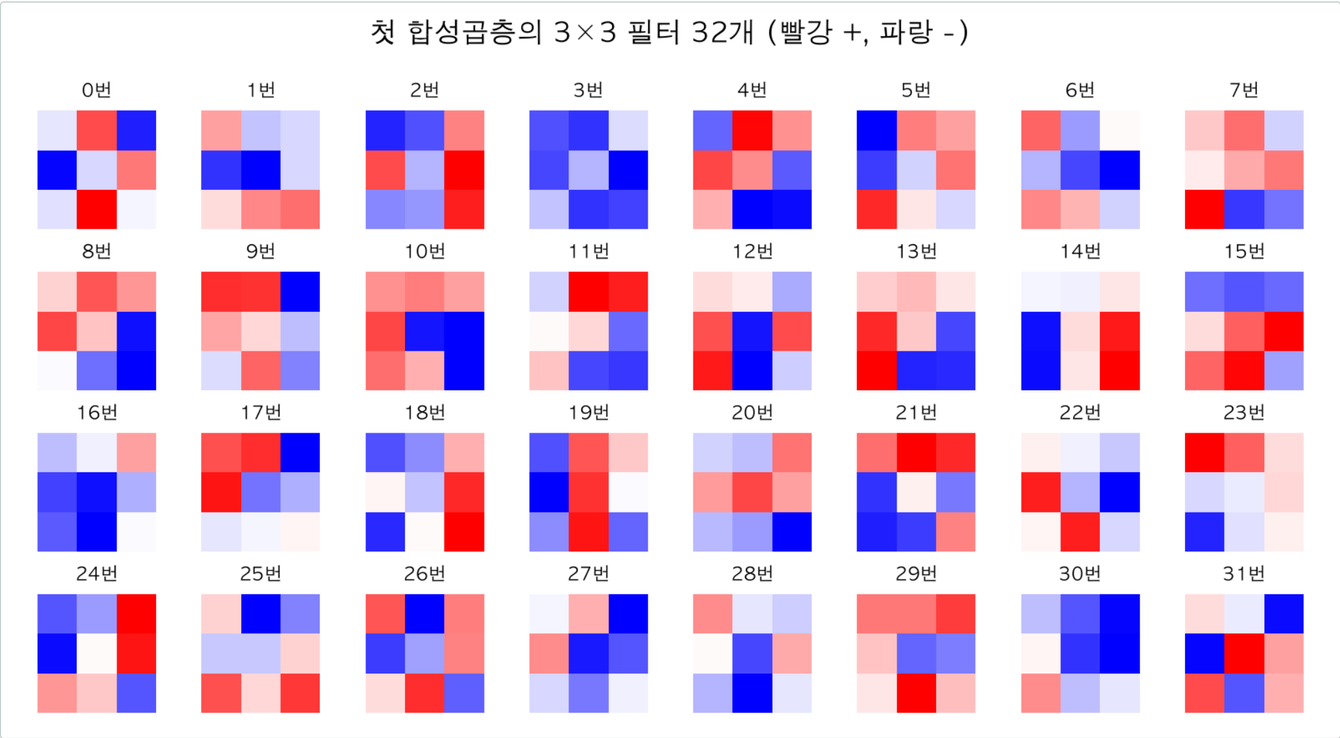


-- 평범한 · — 평범한 + BN · -- 잔차 · — 잔차 + BN
세로는 학습 손실, 가로는 에폭 1~5

5에폭	손실	정확도
평범한	2.302	10.1%
평범한 + BN	0.778	58.7%
잔차	1.153	49.5%
잔차 + BN	0.597	69.2%

평범한 21층은 손실 2.30 = −log(1/10): **찍기**. 덧셈 하나로 살아나고, 실제 ResNet은 **잔차 + BN**

9.9 CNN은 무엇을 보는가: 첫 층의 필터



25 / 32

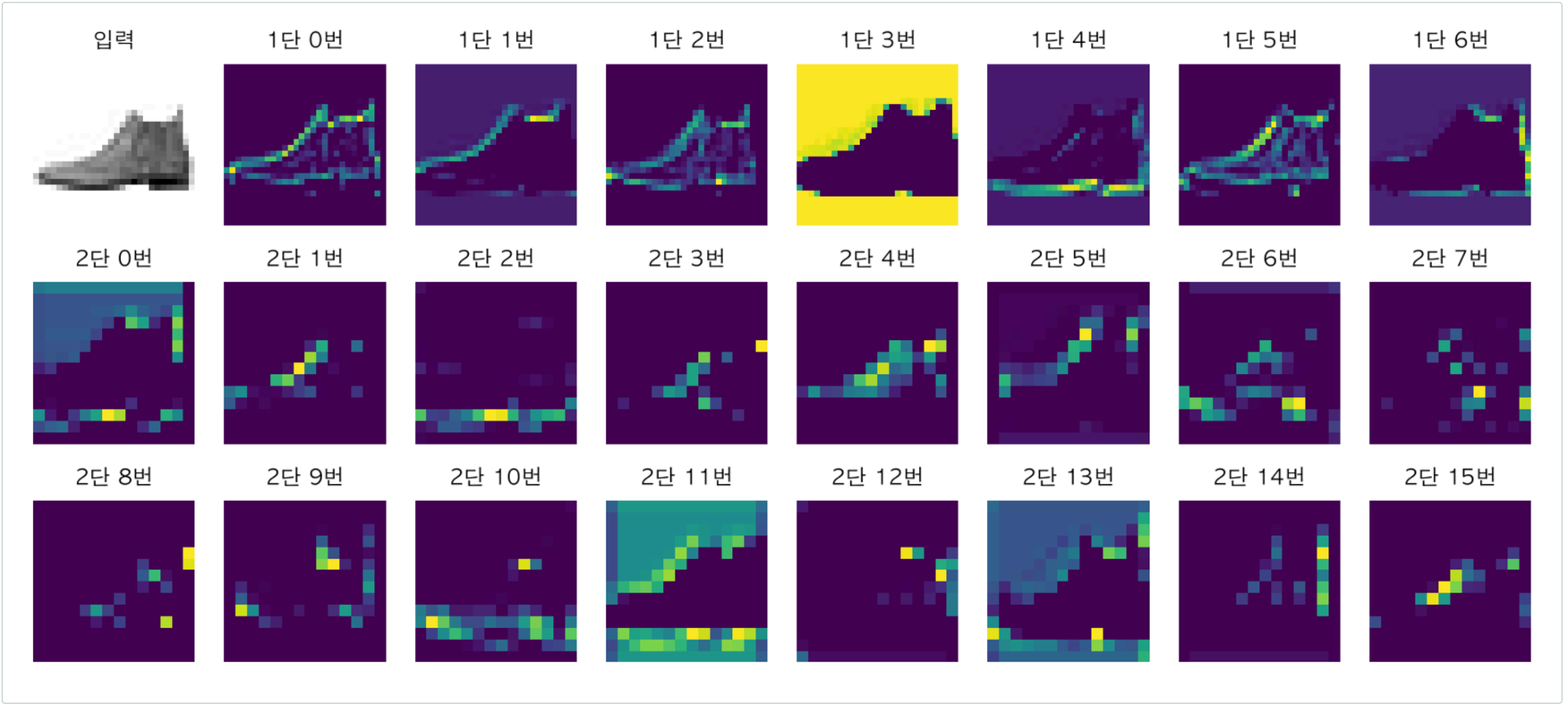
숫자 합이 0에 가까운 **경계형** 필터

잉크에 반응 1개 (29번)
잉크 없는 곳에 반응 6개 (3, 6, 16, 27, 28, 30번)

아무도 "경계를 찾아라"라고 시키지 않았다

19차시 · 9.9 첫 층의 필터 · 예제 09-cnn/b21_filters.py가 만든 cnn_filters.png와 출력 (빨강 +, 파랑 –)

9.9 특징 맵: 층마다 무엇이 켜지나

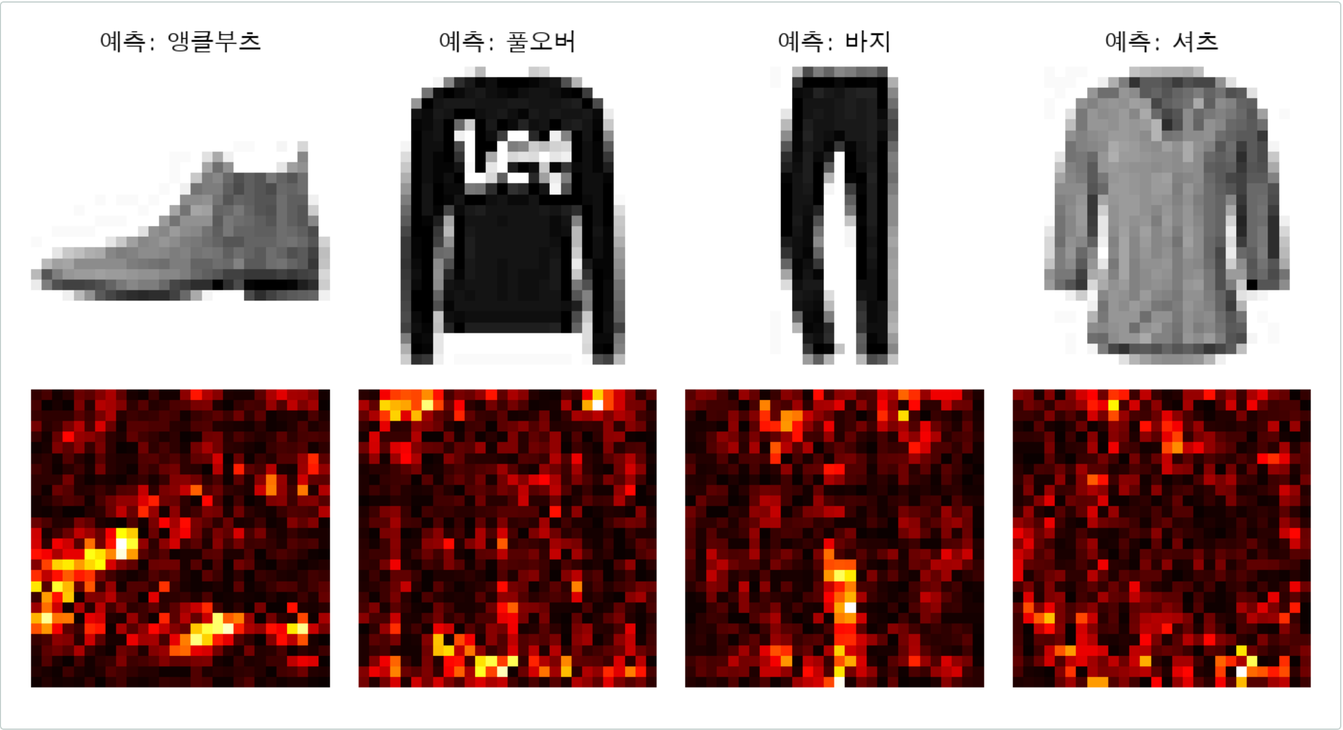


1단 (32, 28, 28): 아직 그림을 닮은 윤곽선

2단 (64, 14, 14): 발등·밑창 같은 **부위 지도**

0인 칸 **50.3% → 79.6%**: 깊을수록 선택적

9.9 어디를 보고 판단했나: 돌출 맵




```
점수[예측].backward()
중요도 = 사진.grad[0, 0].abs()
```

네 장 모두 맞혔고 셔츠만 **93.1%**, 나머지는 99.7% 이상

바지는 **두 다리 사이의 빈틈**이 가장 밝다: "가운데가 갈라졌나"를 스스로 찾았다

잡음이 많아 하나로 단정하지는 않는다

19차시 · 9.9 어디를 보고 판단했나 · 예제 09-cnn/b23_saliency.py가 만든 cnn_saliency.png와 출력 (핵심 두 줄)

직접 실행 10분

1. b17_receptive.py의 for 깊이 in (1, 2, 3)에 4를 더한다. 3×3을 네 번 쌓으면 보는 범위는? 먼저 예상한다
2. b19_grad_flow.py의 for 블록수 in (2, 10, 25)에 50을 더한다 (합성곱 101층). 평범한 CNN과 잔차 CNN의 첫 층 기울기는?
3. b21~b23을 실행하고 만들어진 그림 세 장을 열어, 슬라이드의 그림과 같은지 본다

```
uv run python b19_grad_flow.py
```

19차시 · 직접 실행 · 모두 1~3초 · b21~b23은 18차시의 fashion_cnn.pt가 있어야 돈다

BatchNorm2d 바로 앞의 합성곱을 bias=False로 만드는 이유는?

Q

가 편향이 있으면 학습이 발산해서

나 배치 정규화가 채널 평균을 빼면서 편향을 지워 버려서

다 편향이 있으면 출력 모양이 달라져서

라 ReLU가 편향을 없애 주어서

19차시 정리

합성곱 → BatchNorm2d → ReLU (합성곱은 bias=False), 드롭아웃은 마지막 Linear 앞

VGG는 3×3을 쌓아 **수용 영역**을 넓히고, ResNet은 **x + F(x)** 덧셈 하나로 기울기 소실을 넘었다

첫 층 필터는 대부분 **경계 탐지기**, 깊은 층은 **부위 지도**, 입력 기울기로 판단 근거를 본다

다음 차시: 배운 것을 모두 모아 작은 ResNet으로 옷 분류기를 다시 만든다

20차시

20

실습: 작은 ResNet으로 옷 분류기 다시 만들기

책 9.10절

학습 목표

- 잔차 블록, 전역 평균 풀링, 증강, 검증 분할, 스케줄러를 한 코드로 묶는다
- 시험 정확도와 밀기 결과를 6장 MLP, 9.5절 CNN과 비교한다
- 증강을 끄고 다시 학습해 구조와 데이터의 몫을 가른다

9.10 들어가는 재료 다섯 가지

재료	내용	배운 차시
작은 ResNet	첫 합성곱 + 잔차 블록 셋: 28×28 32채널 → 14×14 64채널 → 7×7 128채널	19차시
전역 평균 풀링 + 드롭아웃	비싼 Flatten + 큰 Linear를 없애고 Linear(128, 10)	17·18·19차시
데이터 증강	RandomCrop(28, padding=2)로 2칸 안에서 무작위로 밀기, 좌우 뒤집기	10차시 (7.6)
검증 데이터 따로	학습 6만 장 중 5,000장. transform만 다른 데이터셋 둘을 같은 번호로 나눈다	11차시 (7.8)
AdamW + OneCycleLR	weight decay, 스케줄러는 배치마다 step()	12·13차시 (8.2, 8.7)

시험 데이터는 설계를 모두 마친 뒤 마지막에 **한 번만** 연다

9.10 모델: 잔차 블록 셋과 전역 평균 풀링

```
def 작은_ResNet() -> nn.Sequential:
    return nn.Sequential(
        nn.Conv2d(1, 32, 3, padding=1, bias=False), nn.BatchNorm2d(32), nn.ReLU(),
        잔차블록(32, 32), 잔차블록(32, 64, 보폭=2), 잔차블록(64, 128, 보폭=2),
        nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Dropout(0.3), nn.Linear(128, 10),
    )
```

크기와 채널

28×28 32채널 → 14×14 64채널 → 7×7 128채널 → 숫자 128개 → 점수 10개

잔차블록

19차시의 블록 그대로. 보폭=2인 블록은 크기를 절반으로, 지름길은 1×1 합성곱

합성곱 뒤 BN

그래서 모든 합성곱이 bias=False. 드롭아웃은 마지막 Linear 앞

9.10 데이터, 분할, 옵티마이저

```
기본 = [v2.ToImage(), v2.ToDtype(torch.float32, scale=True)]
증강 = v2.Compose([v2.ToImage(), v2.RandomCrop(28, padding=2),  # 상하좌우 2칸 안에서 무작위로 밀기
                   v2.RandomHorizontalFlip(), v2.ToDtype(torch.float32, scale=True)])
학습용 = datasets.FashionMNIST("data", train=True, download=True, transform=증강)
검증용 = datasets.FashionMNIST("data", train=True, download=True, transform=v2.Compose(기본))
순서 = torch.randperm(60000, generator=torch.Generator().manual_seed(0))
학습셋, 검증셋 = Subset(학습용, 순서[:55000]), Subset(검증용, 순서[55000:])

에폭수 = 12
옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=2e-3, weight_decay=5e-4)
스케줄러 = torch.optim.lr_scheduler.OneCycleLR(
    옵티마이저, max_lr=3e-3, total_steps=에폭수 * len(학습_로더))
...
        스케줄러.step()                          # OneCycleLR은 배치마다 (8장 8.7절)
```

20차시 · 9.10 실습 · 예제 09-cnn/b24_lab_train.py (핵심 부분, 스케줄러 줄은 화면에 맞게 나눠 적음)

9.10 결과: 시험 정확도 93.04%

```
장치 mps, 파라미터 308,074개
에폭  3: 검증 정확도 85.26%
에폭  6: 검증 정확도 89.78%
에폭  9: 검증 정확도 92.56%
에폭 12: 검증 정확도 93.18%
시험 정확도 93.04%
상의 네 종류: T셔츠 89.9%, 풀오버 91.1%, 코트 88.9%, 셔츠 77.4%
밀기 0~4칸: 93.0%  93.1%  92.9%  92.4%  92.0%
```

파라미터 **308,074**: 9.5절 CNN 421,642보다 적다

초반이 낮은 건 학습률을 올리는 중이고 증강으로 문제가 어려워서

검증 93.18%, 시험 93.04%: 검증 성적을 믿을 수 있었다

88.58% → 91.01% → **93.04%**: 틀리던 사진의 40%쯤(11.42% → 6.96%)을 더 맞힌다

20차시 · 9.10 실습 · 예제 09-cnn/b24_lab_train.py 출력 (이 컴퓨터 MPS, 66초)

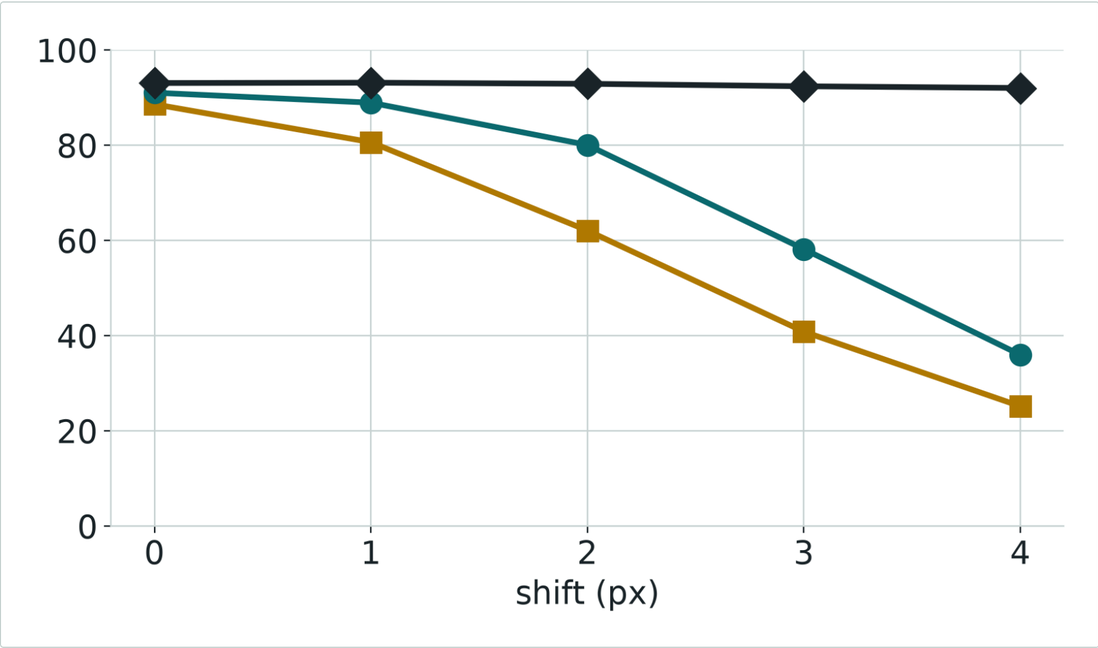
9.10 상의 네 종류: 셔츠만 제자리

모델	T셔츠	풀오버	코트	셔츠
6장 MLP	84.0%	72.9%	87.3%	74.1%
9.5절 CNN	83.7%	89.1%	81.9%	77.9%
9.10절 작은 ResNet	**89.9%**	**91.1%**	**88.9%**	**77.4%**

모델을 바꿔도 셔츠가 안 오르면 **데이터 탓**일 가능성이 크다. 칼라와 단추는 몇 픽셀뿐이고, 사람이 봐도 애매한 사진이 섞여 있다. 모델을 키우기보다 **틀린 사진을 직접 들여다보기**가 먼저

20차시 · 9.10 실습 · 예제 09-cnn/b11_compare.py, b24_lab_train.py 출력에서 상의 네 종류만 모은 표

9.10 네 칸 밀어도 92.0%



■ 6장 MLP · ● 9.5절 CNN · ◆ 작은 ResNet + 증강 · 가로는 민 칸 수

모델	파라미터	4칸 밀기
6장 MLP	235,146	25.2%
9.5절 CNN	421,642	36.0%
작은 ResNet	308,074	**92.0%**

전역 평균 풀링(**구조**)과 무작위 밀기 증강(**데이터**)이 함께 지켜냈다

둘 중 무엇이 얼마나? 직접 실행에서 증강을 꺼 본다

20차시 · 9.10 실습 · 저장된 세 모델 파일로 b13과 같은 밀기 시험을 다시 계산한 그림 (값은 b13, b24 출력과 같다)

직접 실행 10분

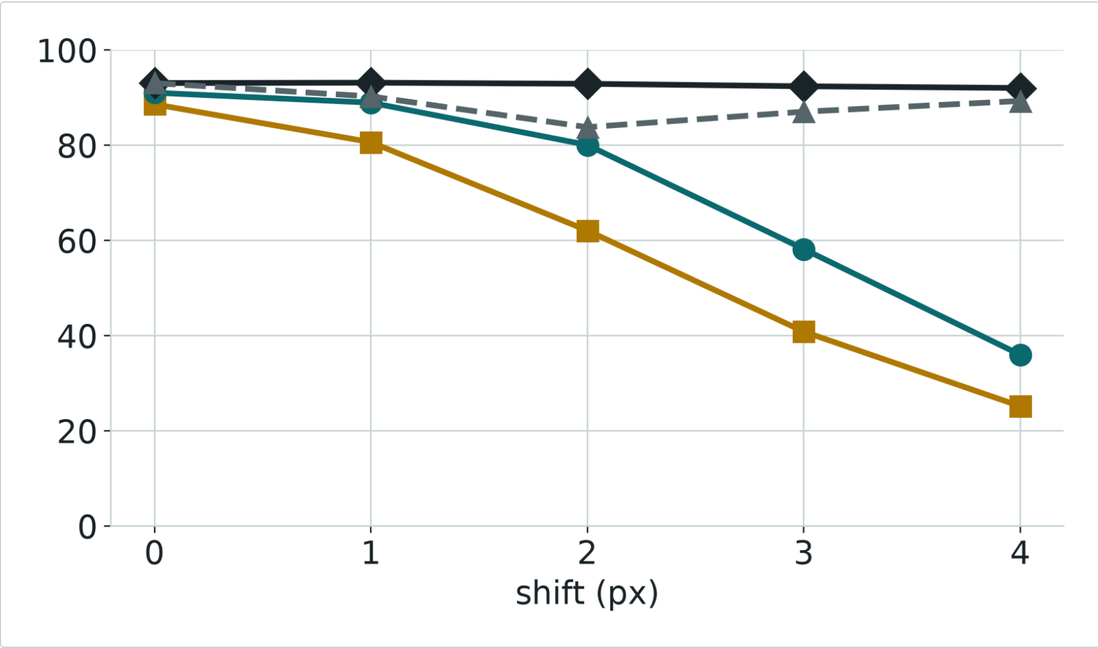
1. 차시 첫머리에 걸어 둔 b24_lab_train.py의 출력을 결과 장의 숫자와 맞춰 본다
2. b24를 복사해 **증강을 끈다**. 두 줄만 바꾸고 실행한다. 시험 정확도와 밀기 0~4칸은 어떻게 될까? 먼저 예상한다

```
증강 = v2.Compose(기본)                          # 실습: 증강을 끈다
torch.save(모델.state_dict(), "resnet_증강없음.pt")  # 책의 모델 파일을 덮어쓰지 않게
```

GPU(MPS): 원래 판 **66초**, 증강 끈 판 **72초**

CPU만 있으면 1에폭에 **119초**: 수업 중엔 강사 화면으로

실습 증강을 끄면: 정확도는 같고 밀기에 약해진다



■ MLP · ● CNN · ◆ ResNet + 증강 · ▲ ResNet 증강 없음 (점선)

작은 ResNet	증강	증강 없음
시험	93.04%	93.02%
밀기 2칸	92.9%	83.8%
밀기 4칸	92.0%	89.3%

구조만으로도 9.5절 CNN(4칸 36.0%)보다 훨씬 버틴다

2칸이 4칸보다 나쁘다: 보폭 2가 두 번이라 4칸 간격이 잘 맞는 것으로 보인다

20차시 · 직접 실행 결과 · b24 실습용 복사본(증강 끔, MPS 72초)의 출력과, 저장된 모델로 다시 잰 밀기 정확도

작은 ResNet이 9.5절 CNN보다 합성곱은 많은데 파라미터는 적은 이유는?

Q

가 잔차 연결이 파라미터를 나눠 써서

나 전역 평균 풀링으로 큰 Linear가 사라져서

다 배치 정규화가 파라미터를 줄여서

라 데이터 증강을 써서

20차시 정리

잔차 블록 + 전역 평균 풀링 + 증강 + 검증 분할 + AdamW·OneCycleLR로 **93.04%**, 파라미터는 더 적게

밀기에 강해진 몫은 **구조**(전역 평균 풀링)와 **데이터**(무작위 밀기 증강)가 나눠 가진다

셔츠처럼 모델을 바꿔도 안 오르는 곳은 **데이터**를 먼저 들여다본다

다음: 4일차 마무리

4일차에 배운 것

1. 합성곱은 **돋보기를 옮기며** 곱하고 더하기, 출력 크기는 공식으로
2. 필터는 채널만큼 두껍고 **어디서나 공유**, 풀링은 요약
3. 첫 CNN 91.01%, 그러나 **Flatten 때문에** 밀면 무너진다
4. 합성곱 → BN → ReLU, 3×3 쌓기, **x + F(x)**, CNN 들여다보기
5. 작은 ResNet + 증강: **93.04%**, 네 칸 밀어도 92.0%

내일 5일차

- 사전학습 모델과 전처리, EuroSAT
- 특징 추출
- 처음부터 학습 vs 미세조정
- 다른 모델, 저장, 실습과 과정 마무리